Python Functions

Class Assignment

Instructor: Tanishq Tyagi | Python Programming

Student Name:	_____	**Date:**	_____
Total Marks:	_____ / 40	**Grade:**	_____

Instructions

- This assignment has **4 activities** worth a total of **40 marks**.
- Write your Python code in the spaces provided or in your IDE and attach printouts.
- Each function must include a docstring describing what it does.
- Test your code with the sample inputs given; include the output in your answer.
- Follow the DRY principle — avoid repeating logic; use helper functions where appropriate.

Activity 1 — Calculate Minimum, Maximum & Average — [10 marks]

Write three functions to analyse a list of numbers. Each function must accept a list of numbers as its parameter and return the result.

Task A — find_minimum(numbers)

Return the smallest number in the list without using Python's built-in min().

> **MARKS** 3 marks for correct logic, return value, and docstring.

Task B — find_maximum(numbers)

Return the largest number in the list without using Python's built-in max().

> **MARKS** 3 marks for correct logic, return value, and docstring.

Task C — calculate_average(numbers)

Return the average (mean) of all numbers in the list, rounded to 2 decimal places.

> **MARKS** 4 marks for correct logic, return value, and docstring.

Starter scaffold:

```
def find_minimum(numbers):
"""Return the smallest number in the list."""
# your code here
pass

def find_maximum(numbers):
"""Return the largest number in the list."""
# your code here
pass

def calculate_average(numbers):
"""Return the average of the list rounded to 2 decimal places."""
# your code here
pass

# Test your functions
data = [45, 12, 78, 3, 56, 29, 91, 7]
print('Min :', find_minimum(data)) # Expected: 3
print('Max :', find_maximum(data)) # Expected: 91
print('Avg :', calculate_average(data)) # Expected: 40.13
```

HINT Loop through the list keeping track of the current minimum/maximum. Initialise your tracker to the first element of the list.

Write your solution below:

Activity 2 — Reusable Age Validation Function — [10 marks]

Build a reusable **validate_age(age)** function that checks whether a given age value is valid for a registration system. The function should demonstrate proper use of **parameters**, **return values**, and **default parameters**.

Task A — validate_age(age, min_age=0, max_age=120)

Accept an age and optional min/max bounds. Return True if the age is a positive integer within [min_age, max_age], else return False.

MARKS 4 marks.

Task B — categorise_age(age)

Use validate_age() inside this function. If invalid, return 'Invalid age'. Otherwise return: 'Child' (0-12), 'Teenager' (13-17), 'Adult' (18-59), 'Senior' (60+).

MARKS 4 marks.

Task C — register_user(name, age)

Use both functions above. Print a welcome message if valid, or a rejection message with the reason if invalid.

MARKS 2 marks.

Sample runs:

```
register_user('Tanishq', 17)
# Output: Welcome Tanishq! Category: Teenager

register_user('Bot', -5)
# Output: Registration failed for Bot. Reason: Invalid age

register_user('Senior', 65)
# Output: Welcome Senior! Category: Senior
```

HINT Call validate_age() from inside categorise_age() — don't repeat the validation logic. This is the DRY principle in action!

Write your solution below:

Activity 3 — Function-Based Calculator [10 marks]

Build a modular calculator using separate functions for each operation. This activity focuses on **keyword arguments**, **return values**, and combining multiple small functions.

Task A — Core operation functions

Write four functions: add(a, b), subtract(a, b), multiply(a, b), divide(a, b). Each returns the result. divide() must handle division by zero by returning None and printing a warning.

MARKS 4 marks.

Task B — calculate(a, b, operation='add')

A dispatcher function that takes two numbers and an operation name as a keyword argument (default 'add'). Call the appropriate function based on the operation string. Return the result.

MARKS 4 marks.

Task C — Keyword argument demo

Call calculate() three times using keyword arguments in different orders. Print the results. Show that keyword arguments allow you to skip the default.

MARKS 2 marks.

Starter scaffold:

```
def add(a, b): return a + b
def subtract(a, b): return a - b
def multiply(a, b): # your code
def divide(a, b): # your code — handle zero!

def calculate(a, b, operation='add'):
"""Dispatch to the correct operation function."""
# your code here
pass

# Task C — keyword argument calls
print(calculate(10, 5, operation='subtract')) # 5
print(calculate(b=4, a=3, operation='multiply')) # 12
print(calculate(20, 4)) # 24 (default add)
```

HINT Use a dictionary mapping operation names to functions, or a series of if/elif checks inside calculate().

Write your solution below:

Activity 4 — Student Marks Analyser — [10 marks]

Build a student marks analyser that **returns multiple values** and uses **tuple unpacking**. This also requires careful use of **variable scope** and the **LEGB rule**.

Task A — analyse_marks(marks)

Accept a list of marks. Return three values: total, average (2 d.p.), grade. Grade scale: A (>=90), B (>=75), C (>=60), D (>=45), F (<45). Use tuple unpacking when calling this function.

MARKS 5 marks.

Task B — print_report(name, marks)

Call analyse_marks() inside this function and unpack all three return values. Print a formatted report card showing name, all individual marks, total, average, and grade.

MARKS 3 marks.

Task C — Scope question (written answer)

In the space below, explain what would happen if you defined a variable 'total = 0' at the top of your script AND inside analyse_marks(). Which value does the function use, and why? Reference the

LEGB rule in your answer.

MARKS 2 marks.

Starter scaffold + test data:

```
def analyse_marks(marks):
"""Return (total, average, grade) for a list of marks."""
# your code here
pass

def print_report(name, marks):
"""Print a formatted report card for the student."""
total, average, grade = analyse_marks(marks) # unpack!
# your code here
pass

# Test
student1_marks = [88, 92, 76, 95, 84]
student2_marks = [55, 42, 60, 48, 38]
print_report('Shachi', student1_marks)
print_report('Mourya', student2_marks)
```

Expected output for Shachi:

```
================================
REPORT CARD — Shachi
================================
Marks : 88 92 76 95 84
Total : 435
Average : 87.0
Grade : B
================================
```

HINT Use sum() and len() for total and count. Build the grade using if/elif. Remember: return total, average, grade returns a tuple.

Task C — Written answer (scope explanation):

_

_

_

_

_

_

_

Python Functions Assignment | Tanishq Tyagi | Total: 40 marks